Aplicação do Teorema Mestre à Análise de Algoritmos Recursivos

Três problemas, um para cada caso do teorema: análise teórica, demonstração por indução e validação experimental

Autores: Vitor Kenzo Matsunaga, Gustavo Alves, Matheus Brunes

Disciplina: Análise de Algoritmos

Professor: Zoe Roberto

Outubro de 2026

Sumário

1 Introdução **3**

2 Revisão Teórica **3**

2.1 O Teorema Mestre . . . 3

2.2 Os três casos . . . 4

2.3 Observações sobre a aplicação . . . 4

3 Metodologia **4**

3.1 Objetivo . . . 4

3.2 Ferramentas utilizadas . . . 5

3.3 Métodos . . . 5

3.4 Justificativa das escolhas . . . 6

3.5 Limitações e fraquezas da metodologia . . . 6

4 Análise Teórica **7**

4.1 Problema 1 — Caso 1: multiplicação de polinômios por Karatsuba . . . 7

4.1.1 Descrição do problema . . . 7

4.1.2 A recorrência analisada . . . 7

4.1.3 Valores de a, b e $f(n)$, e cálculo de $n^{\log_b a}$ . . . 7

4.1.4 Enquadramento e justificativa . . . 8

4.1.5 Complexidade assintótica . . . 8

4.1.6 Demonstração por indução . . . 8

4.2 Problema 2 — Caso 2: MergeSort ternário . . . 9

4.2.1 Descrição do problema . . . 9

4.2.2 A recorrência analisada . . . 9

4.2.3 Valores de a, b e $f(n)$, e cálculo de $n^{\log_b a}$ . . . 9

4.2.4 Enquadramento e justificativa . . . 9

4.2.5 Complexidade assintótica . . . 10

4.2.6 Demonstração por indução . . . 10

4.3 Problema 3 — Caso 3: par mais próximo em uma dimensão . . . 10

4.3.1 Descrição do problema . . . 10

4.3.2 A recorrência analisada . . . 11

4.3.3 Valores de a, b e $f(n)$, e cálculo de $n^{\log_b a}$ . . . 11

4.3.4 Enquadramento e justificativa . . . 11

4.3.5 Complexidade assintótica . . . 11

4.3.6 Demonstração por indução . . . 11

5 Resultados Experimentais e Comparação com a Teoria **12**

5.1 Problema 1 — Caso 1: Karatsuba . . . 13

5.1.1 Dados coletados . . . 13

5.1.2 Análise e comparação com a teoria . . . 14

5.1.3 Comparação com o método direto . . . 14

5.2 Problema 2 — Caso 2: MergeSort ternário . . . 15

5.2.1 Dados coletados . . . 15

5.2.2 Análise e comparação com a teoria 16
5.3 Problema 3 — Caso 3: par mais próximo 18
5.3.1 Dados coletados 18
5.3.2 Análise e comparação com a teoria 18
5.4 Comparação entre os três casos 20
5.5 Custo real de cada operação elementar 20
5.6 Síntese das convergências e divergências 22

6 Conclusão **22**
6.1 Resumo dos resultados 22
6.2 Síntese da comparação entre teoria e prática 23
6.3 Principais observações 23
6.4 Limitações encontradas 23
6.5 Considerações adicionais 23

A Código-fonte **24**

B Arquivo de resultados **30**

1 Introdução

Este trabalho aplica o Teorema Mestre à análise de três algoritmos recursivos distintos, escolhidos de modo que cada um se enquadre em um caso diferente do teorema, valida os resultados obtidos por demonstração matemática e os confronta com medições experimentais.

Os objetivos são:

1. escolher três recorrências do tipo $T(n) = a\,T(n/b) + f(n)$ que se enquadrem, respectivamente, nos Casos 1, 2 e 3 do Teorema Mestre, com $a \neq 2$ nos dois primeiros e $a \neq b$ em pelo menos um deles;
2. implementar, em linguagem C, uma solução recursiva para cada problema, instrumentada com um contador de operações elementares e capaz de executar para diferentes tamanhos de entrada, registrando todos os experimentos em um único arquivo CSV;
3. realizar a análise teórica de cada recorrência, identificando a, b e $f(n)$, calculando $n^{\log_b a}$, enquadrando a recorrência no caso correto e determinando a complexidade assintótica em notação Θ;
4. demonstrar formalmente, por indução matemática, a forma fechada de cada recorrência, verificando que o resultado coincide com o previsto pelo Teorema Mestre;
5. comparar as previsões teóricas com os dados experimentais, por meio de tabelas e gráficos, e discutir as convergências e divergências observadas.

Os três problemas escolhidos são apresentados na Tabela 1.

Tabela 1: *Os três problemas analisados e o enquadramento de cada um.*

	Algoritmo	a	b	$f(n)$	Caso e complexidade
Problema 1	Karatsuba (produto de polinômios)	3	2	$\Theta(n)$	Caso 1 — $\Theta(n^{\log_2 3})$
Problema 2	MergeSort ternário	3	3	$\Theta(n)$	Caso 2 — $\Theta(n \lg n)$
Problema 3	Par mais próximo em 1D	2	2	$\Theta(n^2)$	Caso 3 — $\Theta(n^2)$

As restrições do enunciado são atendidas: nos Problemas 1 e 2 tem-se $a = 3 \neq 2$, e no Problema 1 ocorre $a = 3 \neq b = 2$. Os três problemas são distintos entre si e todos admitem implementação recursiva direta em C.

Adianta-se o resultado principal: nas 37 execuções registradas, o número de operações elementares medido coincidiu *exatamente* com a forma fechada demonstrada por indução para os três problemas, sem nenhuma diferença, e as taxas de crescimento medidas convergiram para os valores previstos pelo Teorema Mestre.

2 Revisão Teórica

2.1 O Teorema Mestre

O Teorema Mestre fornece a solução assintótica de recorrências da forma

$$T(n) = a\,T\Big(\frac{n}{b}\Big) + f(n), \tag{1}$$

com $a \geq 1$ e $b > 1$ constantes e $f(n)$ uma função assintoticamente positiva. A recorrência descreve um algoritmo de divisão e conquista que quebra um problema de tamanho n em a subproblemas de tamanho n/b, resolve cada um recursivamente e gasta $f(n)$ para dividir o problema e combinar as soluções parciais.

A comparação decisiva é entre $f(n)$, o custo fora da recursão, e a função

$$n^{\log_b a},$$

que mede o custo acumulado nas folhas da árvore de recursão. De fato, a árvore tem $\log_b n$ níveis e, no nível i, há a^i subproblemas de tamanho n/b^i; no último nível há $a^{\log_b n} = n^{\log_b a}$ folhas. O teorema distingue três situações, conforme qual dos dois custos domina.

2.2 Os três casos

Caso 1 — as folhas dominam. Se $f(n) = O\big(n^{\log_b a - \varepsilon}\big)$ para alguma constante $\varepsilon > 0$, então

$$T(n) = \Theta\Big(n^{\log_b a}\Big).$$

O custo de dividir e combinar é polinomialmente menor que o custo das folhas; o trabalho cresce a cada nível descido na árvore, e o somatório é dominado pelo último nível.

Caso 2 — todos os níveis custam o mesmo. Se $f(n) = \Theta\big(n^{\log_b a}\big)$, então

$$T(n) = \Theta\Big(n^{\log_b a} \lg n\Big).$$

Cada um dos $\Theta(\lg n)$ níveis da árvore contribui com o mesmo custo $\Theta\big(n^{\log_b a}\big)$, e daí o fator logarítmico adicional.

Caso 3 — a raiz domina. Se $f(n) = \Omega\big(n^{\log_b a + \varepsilon}\big)$ para alguma constante $\varepsilon > 0$, e se além disso vale a *condição de regularidade*

$$a\, f\Big(\frac{n}{b}\Big) \le c\, f(n) \quad \text{para alguma constante } c < 1 \text{ e todo } n \text{ suficientemente grande,} \tag{2}$$

então

$$T(n) = \Theta\big(f(n)\big).$$

O custo de dividir e combinar é polinomialmente maior que o das folhas, e a condição de regularidade garante que esse custo decresce geometricamente a cada nível, de modo que a soma é dominada pela raiz.

2.3 Observações sobre a aplicação

Duas ressalvas são relevantes para este trabalho. Primeiro, o teorema não cobre todos os casos possíveis: existem lacunas entre os casos, quando $f(n)$ é menor que $n^{\log_b a}$ mas não *polinomialmente* menor (por exemplo $f(n) = n^{\log_b a}/\lg n$), e nelas o teorema não se aplica. Segundo, a forma (1) ignora os pisos e tetos que surgem quando n não é divisível por b; demonstra-se que isso não altera o resultado assintótico, e neste trabalho a questão foi contornada escolhendo tamanhos de entrada que são potências exatas de b nos Problemas 1 e 2, e obtendo uma forma fechada válida para todo n no Problema 3.

Por fim, o Teorema Mestre fornece apenas a *classe* assintótica. As demonstrações por indução da Seção 4 vão além e estabelecem a forma fechada exata de cada recorrência, o que permite comparar teoria e experimento número a número, e não apenas por ordem de grandeza.

3 Metodologia

3.1 Objetivo

O objetivo geral é verificar, de forma teórica e experimental, que a complexidade prevista pelo Teorema Mestre descreve o comportamento real de implementações recursivas. Os objetivos específicos são:

- **Problema 1:** verificar que a multiplicação de polinômios por Karatsuba, com recorrência enquadrada no Caso 1, tem custo $\Theta(n^{\log_2 3})$, e observar a partir de que tamanho ela supera o método direto $\Theta(n^2)$;
- **Problema 2:** verificar que o MergeSort ternário, enquadrado no Caso 2, tem custo $\Theta(n \lg n)$, e isolar experimentalmente o fator logarítmico;
- **Problema 3:** verificar que o algoritmo do par mais próximo com combinação por força bruta, enquadrado no Caso 3, tem custo $\Theta(n^2)$, ou seja, que o custo da combinação domina o das chamadas recursivas.

3.2 Ferramentas utilizadas

Implementação

Linguagem C, padrão C99, compilada com GCC 13.3.0 e as opções `-O2 -Wall -Wextra -std=c99`.

Medição de desempenho

Duas grandezas foram medidas. A primeira é um **contador de operações elementares**, uma variável global incrementada exatamente uma vez por operação elementar; a definição da operação elementar é dada no início de cada problema. A segunda é o **tempo de parede**, obtido por `clock_gettime(CLOCK_MONOTONIC)` em Linux e `QueryPerformanceCounter` em Windows, ambos relógios monotônicos de alta resolução.

Registro dos dados

Um único arquivo CSV, `resultados/resultados.csv`, recebe uma linha por execução, com o número do problema, o algoritmo, a recorrência, o tamanho da entrada, as operações contadas, o valor previsto pela forma fechada, a diferença entre os dois, o tempo mediano e o número de repetições.

Análise, tabelas e gráficos

Python 3.13 com `pandas` 3.0 (leitura e agregação dos dados), `numpy` 2.5 (ajustes por mínimos quadrados) e `matplotlib` 3.11 (geração dos gráficos). As tabelas deste relatório foram geradas automaticamente a partir do mesmo CSV.

Ambiente

Linux x86-64, processador Intel Xeon de 2,10 GHz.

3.3 Métodos

Coleta dos dados. Para cada problema, o programa executa o algoritmo em uma sequência crescente de tamanhos de entrada: potências de 2 de 2 a 4.096 no Problema 1, potências de 3 de 3 a 531.441 no Problema 2 e potências de 2 de 2 a 8.192 no Problema 3. Em cada tamanho, a execução é repetida 15 vezes, e registra-se o *tempo mediano* dessas repetições. As entradas são geradas por um gerador congruencial linear próprio, com semente fixa, de modo que os mesmos dados sejam produzidos em qualquer máquina ou compilador. O contador de operações é zerado antes de cada execução.

Análise. Para cada tamanho, compara-se o valor do contador com o valor previsto pela forma fechada demonstrada na Seção 4; a diferença entre os dois é registrada no CSV. Para verificar a classe assintótica, usam-se duas grandezas independentes da constante multiplicativa: a **razão de crescimento** $T(bn)/T(n)$, que deve convergir para a, e a **inclinação do ajuste log-log**, que deve convergir para o expoente $\log_b a$ previsto. As mesmas grandezas são calculadas sobre os tempos medidos.

3.4 Justificativa das escolhas

A contagem de operações elementares é o instrumento central deste trabalho, e a escolha é deliberada. O Teorema Mestre é um resultado sobre o *número de operações* de um algoritmo, não sobre o tempo de relógio: ele raciocina em um modelo de custo uniforme, em que toda operação elementar custa o mesmo. Um contador mede exatamente essa grandeza, e por isso permite uma comparação sem erro de medição, enquanto o tempo está sujeito a escalonamento de processos, hierarquia de memória e variação de frequência do processador. Além disso, as operações elementares foram definidas, nos três problemas, de modo a não dependerem dos *valores* da entrada, apenas do seu tamanho, o que torna cada medição determinística e reprodutível.

O tempo de parede não foi abandonado, porém: ele é a grandeza que realmente interessa na prática, e é justamente a comparação entre as duas que revela onde o modelo teórico se afasta da máquina real (Seção 5.5). O uso da mediana de 15 repetições, em vez da média, reduz o efeito de execuções atípicas causadas por interferência do sistema operacional.

A escolha de tamanhos que são potências exatas de b nos Problemas 1 e 2 elimina os pisos e tetos das recorrências, permitindo uma forma fechada exata e uma demonstração por indução limpa. Os intervalos de tamanho foram escolhidos de modo que o maior caso de cada problema leve dezenas de milissegundos, tempo suficiente para que a medição seja confiável sem que a bateria completa se torne demorada.

3.5 Limitações e fraquezas da metodologia

- **O modelo de custo uniforme não vale na máquina real.** O Teorema Mestre supõe que toda operação elementar custa o mesmo, mas o custo real de um acesso à memória depende de o dado estar ou não em *cache*. Isso é visível nos dados (Seção 5.5) e é a principal causa das divergências entre a curva de operações e a de tempo.
- **Operações não contabilizadas.** O contador registra apenas as operações elementares definidas. Chamadas de função, alocações com `malloc` e `free`, cópias de memória e aritmética de índices não são contadas, embora consumam tempo. No Problema 1, em particular, cada chamada recursiva faz cinco alocações, o que pesa no tempo mas não no contador.
- **Ruído de medição em entradas pequenas.** Para os menores tamanhos, o tempo de execução é da ordem de dezenas de nanossegundos, comparável à própria resolução e ao custo da chamada ao relógio. Por isso, os ajustes de inclinação sobre os tempos foram calculados apenas sobre os pontos maiores.
- **Efeito do compilador.** Com `-O2`, o GCC pode vetorizar laços, eliminar código redundante ou converter recursões de cauda em laços. Isso altera o tempo medido, mas não o número de operações contadas, o que é mais um argumento a favor do contador como instrumento principal.
- **Faixa limitada de tamanhos.** Os maiores tamanhos foram limitados pela memória e pelo tempo de execução, especialmente no Problema 3, cujo custo quadrático torna proibitivo ir muito além de $n = 8.192$. As conclusões assintóticas são, portanto, uma extrapolação a partir de uma faixa finita.
- **Validade restrita das formas fechadas exatas.** Nos Problemas 1 e 2, as formas fechadas demonstradas valem para n potência exata de b; para outros valores de n, valem a recorrência com pisos e tetos e a classe assintótica, mas não a fórmula exata. No Problema 3, a forma fechada vale para todo $n \geq 1$.

4 Análise Teórica

4.1 Problema 1 — Caso 1: multiplicação de polinômios por Karatsuba

4.1.1 Descrição do problema

Dados dois polinômios de n coeficientes cada,

$$P(x) = \sum_{i=0}^{n-1} p_i x^i, \qquad Q(x) = \sum_{i=0}^{n-1} q_i x^i,$$

deseja-se calcular o produto $P(x)\,Q(x)$, que tem $2n-1$ coeficientes. O método direto calcula cada produto $p_i q_j$, o que custa $\Theta(n^2)$.

O algoritmo de Karatsuba divide cada polinômio ao meio, com $m = n/2$:

$$P = P_0 + P_1 x^m, \qquad Q = Q_0 + Q_1 x^m,$$

em que P_0, P_1, Q_0, Q_1 têm m coeficientes. O produto seria, ingenuamente,

$$PQ = P_0Q_0 + (P_0Q_1 + P_1Q_0)x^m + P_1Q_1 x^{2m},$$

o que exigiria quatro multiplicações de tamanho m. A observação de Karatsuba é que o termo do meio pode ser obtido a partir de um único produto adicional:

$$P_0Q_1 + P_1Q_0 = (P_0 + P_1)(Q_0 + Q_1) - P_0Q_0 - P_1Q_1.$$

São necessárias, portanto, apenas *três* multiplicações de tamanho m:

$$z_0 = P_0Q_0, \qquad z_2 = P_1Q_1, \qquad z_1 = (P_0 + P_1)(Q_0 + Q_1) - z_0 - z_2,$$

e o produto é $PQ = z_0 + z_1 x^m + z_2 x^{2m}$.

Operação elementar contada: cada soma, subtração ou multiplicação de coeficientes. Cópias de memória não são contadas. A contagem independe dos valores dos coeficientes.

4.1.2 A recorrência analisada

Com $m = n/2$, o trabalho fora das chamadas recursivas é: m somas para obter $P_0 + P_1$ e m para $Q_0 + Q_1$; $2(2m-1)$ subtrações para $z_1 = (P_0 + P_1)(Q_0 + Q_1) - z_0 - z_2$; e $3(2m-1)$ somas para acumular z_0, z_1 e z_2 nas posições corretas do resultado. O total é

$$2m + 2(2m-1) + 3(2m-1) = 12m - 5 = 6n - 5.$$

No caso base, $n = 1$, há uma única multiplicação. A recorrência é, portanto,

$$T(1) = 1, \qquad T(n) = 3\,T\!\left(\frac{n}{2}\right) + 6n - 5 \quad \text{para } n \geq 2. \tag{3}$$

4.1.3 Valores de a, b e $f(n)$, e cálculo de $n^{\log_b a}$

$$a = 3, \qquad b = 2, \qquad f(n) = 6n - 5 = \Theta(n).$$

$$n^{\log_b a} = n^{\log_2 3}, \qquad \log_2 3 = \frac{\ln 3}{\ln 2} \approx 1{,}584963,$$

ou seja, $n^{\log_2 3} \approx n^{1{,}5850}$.

Note que $a = 3 \neq 2$ e $a = 3 \neq b = 2$, o que atende às duas restrições do enunciado.

4.1.4 Enquadramento e justificativa

Compara-se $f(n) = \Theta(n) = \Theta(n^1)$ com $n^{\log_2 3} \approx n^{1{,}5850}$. Como $1 < \log_2 3$, a função f é polinomialmente menor, o que caracteriza o **Caso 1**. Formalmente, é preciso exibir uma constante $\varepsilon > 0$ tal que $f(n) = O\big(n^{\log_2 3 - \varepsilon}\big)$. Tomando

$$\varepsilon = \log_2 3 - 1 \approx 0{,}584963 > 0,$$

tem-se $n^{\log_2 3 - \varepsilon} = n^1 = n$, e de fato $f(n) = 6n - 5 \leq 6n$ para todo $n \geq 1$, ou seja, $f(n) = O(n)$. A condição do Caso 1 está satisfeita.

4.1.5 Complexidade assintótica

Pelo Caso 1 do Teorema Mestre,

$$\boxed{T(n) = \Theta\Big(n^{\log_2 3}\Big) \approx \Theta\big(n^{1{,}5850}\big)\,.}$$

4.1.6 Demonstração por indução

Proposição 1. *Para todo inteiro $k \geq 0$, a solução da recorrência* (3) *em $n = 2^k$ é*

$$T(2^k) = \frac{21 \cdot 3^k - 24 \cdot 2^k + 5}{2}.$$

Demonstração. Por indução sobre k.

Caso base ($k = 0$, isto é, $n = 1$). A recorrência dá $T(1) = 1$, e a fórmula dá

$$\frac{21 \cdot 3^0 - 24 \cdot 2^0 + 5}{2} = \frac{21 - 24 + 5}{2} = \frac{2}{2} = 1.$$

Os dois valores coincidem.

Hipótese de indução. Suponha que, para algum $k \geq 0$,

$$T(2^k) = \frac{21 \cdot 3^k - 24 \cdot 2^k + 5}{2}.$$

Passo indutivo. Aplicando a recorrência (3) com $n = 2^{k+1}$, cujo metade é 2^k:

$$\begin{aligned}
T(2^{k+1}) &= 3\,T(2^k) + 6 \cdot 2^{k+1} - 5 \\
&= 3 \cdot \frac{21 \cdot 3^k - 24 \cdot 2^k + 5}{2} + 12 \cdot 2^k - 5 \\
&= \frac{63 \cdot 3^k - 72 \cdot 2^k + 15}{2} + \frac{24 \cdot 2^k - 10}{2} \\
&= \frac{63 \cdot 3^k - 48 \cdot 2^k + 5}{2} \\
&= \frac{21 \cdot (3 \cdot 3^k) - 24 \cdot (2 \cdot 2^k) + 5}{2} \\
&= \frac{21 \cdot 3^{k+1} - 24 \cdot 2^{k+1} + 5}{2},
\end{aligned}$$

que é precisamente a fórmula avaliada em $k + 1$.

Pelo princípio da indução matemática, a fórmula vale para todo $k \geq 0$. □

Coincidência com o Teorema Mestre. Substituindo $n = 2^k$, tem-se $k = \log_2 n$ e

$$3^k = 3^{\log_2 n} = \left(2^{\log_2 3}\right)^{\log_2 n} = n^{\log_2 3},$$

de modo que

$$T(n) = \frac{21\, n^{\log_2 3} - 24\, n + 5}{2} = 10{,}5\, n^{\log_2 3} - 12\, n + 2{,}5. \tag{4}$$

O termo dominante é $10{,}5\, n^{\log_2 3}$, e os termos restantes são de ordem estritamente menor, logo $T(n) = \Theta\big(n^{\log_2 3}\big)$, exatamente o resultado obtido pelo Teorema Mestre.

4.2 Problema 2 — Caso 2: MergeSort ternário

4.2.1 Descrição do problema

Dado um vetor de n elementos, deseja-se ordená-lo em ordem não decrescente. O MergeSort ternário divide o vetor em *três* blocos de aproximadamente $n/3$ elementos cada, ordena os três recursivamente e intercala os três blocos ordenados em uma única passagem, escolhendo a cada passo o menor entre os primeiros elementos ainda não consumidos dos três blocos.

Operação elementar contada: cada posicionamento de um elemento no vetor de saída durante a intercalação. Como a intercalação de três blocos que somam n elementos posiciona exatamente n elementos, independentemente dos valores, a contagem é determinística e depende apenas de n.

4.2.2 A recorrência analisada

Cada nível da recursão faz três chamadas sobre blocos de tamanho $n/3$ e executa uma intercalação que posiciona n elementos. Um vetor com 0 ou 1 elemento já está ordenado e não gera intercalação. Logo,

$$T(1) = T(0) = 0, \qquad T(n) = 3\,T\Big(\frac{n}{3}\Big) + n \quad \text{para } n \geq 2. \tag{5}$$

4.2.3 Valores de a, b e $f(n)$, e cálculo de $n^{\log_b a}$

$$a = 3, \qquad b = 3, \qquad f(n) = n = \Theta(n).$$
$$n^{\log_b a} = n^{\log_3 3} = n^1 = n.$$

Note que $a = 3 \neq 2$, conforme exige o enunciado para o Problema 2.

4.2.4 Enquadramento e justificativa

Compara-se $f(n) = n$ com $n^{\log_3 3} = n$. As duas funções são iguais, logo em particular

$$f(n) = \Theta\Big(n^{\log_3 3}\Big),$$

com constantes $c_1 = c_2 = 1$ e $n_0 = 1$, o que caracteriza o **Caso 2**. Interpretando pela árvore de recursão: no nível i há 3^i subproblemas de tamanho $n/3^i$, e o custo de combinação de cada um é $n/3^i$; o custo total do nível é $3^i \cdot (n/3^i) = n$, o mesmo em todos os níveis. Como a árvore tem $\log_3 n$ níveis, o custo total é $n \log_3 n$.

Observe que nem o Caso 1 nem o Caso 3 se aplicam: não existe $\varepsilon > 0$ tal que $n = O(n^{1-\varepsilon})$ nem tal que $n = \Omega(n^{1+\varepsilon})$.

4.2.5 Complexidade assintótica

Pelo Caso 2 do Teorema Mestre,

$$\boxed{T(n) = \Theta\Big(n^{\log_3 3} \lg n\Big) = \Theta(n \lg n).}$$

A base do logaritmo é irrelevante na notação assintótica, pois $\log_3 n = \lg n / \lg 3$ difere de $\lg n$ apenas por uma constante multiplicativa.

4.2.6 Demonstração por indução

Proposição 2. *Para todo inteiro $k \geq 0$, a solução da recorrência* (5) *em $n = 3^k$ é*

$$T(3^k) = k \cdot 3^k, \qquad \textit{isto é,} \qquad T(n) = n \log_3 n.$$

Demonstração. Por indução sobre k.

Caso base ($k = 0$, isto é, $n = 1$). A recorrência dá $T(1) = 0$, e a fórmula dá $0 \cdot 3^0 = 0$. Os dois valores coincidem.

Hipótese de indução. Suponha que, para algum $k \geq 0$, valha $T(3^k) = k \cdot 3^k$.

Passo indutivo. Aplicando a recorrência (5) com $n = 3^{k+1}$, cujo terço é 3^k:

$$\begin{aligned} T(3^{k+1}) &= 3\,T(3^k) + 3^{k+1} \\ &= 3 \cdot k \cdot 3^k + 3^{k+1} \\ &= k \cdot 3^{k+1} + 3^{k+1} \\ &= (k+1) \cdot 3^{k+1}, \end{aligned}$$

que é precisamente a fórmula avaliada em $k + 1$.

Pelo princípio da indução matemática, a fórmula vale para todo $k \geq 0$. Substituindo $k = \log_3 n$, obtém-se $T(n) = n \log_3 n$. □

Coincidência com o Teorema Mestre. Como $\log_3 n = \lg n / \lg 3$, tem-se

$$T(n) = n \log_3 n = \frac{1}{\lg 3}\, n \lg n \approx 0{,}6309\, n \lg n, \tag{6}$$

isto é, $T(n) = \Theta(n \lg n)$, exatamente o resultado obtido pelo Teorema Mestre.

4.3 Problema 3 — Caso 3: par mais próximo em uma dimensão

4.3.1 Descrição do problema

Dado um vetor v com n números, deseja-se encontrar a menor distância $|v_i - v_j|$ entre dois elementos distintos. A solução analisada é de divisão e conquista com etapa de combinação por força bruta: o vetor é dividido ao meio, a menor distância de cada metade é obtida recursivamente, e a etapa de combinação examina *todos* os pares formados por um elemento da metade esquerda e um da direita, atualizando o mínimo.

Este é deliberadamente um algoritmo ineficiente para o problema — ordenar o vetor e comparar vizinhos consecutivos resolveria em $\Theta(n \lg n)$. A escolha é intencional: trata-se de um algoritmo recursivo genuíno cuja etapa de combinação é cara o suficiente para dominar as chamadas recursivas, que é precisamente a situação do Caso 3.

Operação elementar contada: cada comparação de distância entre dois elementos na etapa de combinação. A contagem independe dos valores do vetor.

4.3.2 A recorrência analisada

Dividindo o intervalo de n elementos em uma metade esquerda com $\lfloor n/2 \rfloor$ elementos e uma direita com $\lceil n/2 \rceil$, a etapa de combinação examina todos os pares cruzados, ou seja, $\lfloor n/2 \rfloor \cdot \lceil n/2 \rceil$ pares. Um vetor com um único elemento não contém par algum. A recorrência exata é

$$T(1) = 0, \qquad T(n) = T\Big(\Big\lfloor \frac{n}{2} \Big\rfloor\Big) + T\Big(\Big\lceil \frac{n}{2} \Big\rceil\Big) + \Big\lfloor \frac{n}{2} \Big\rfloor \cdot \Big\lceil \frac{n}{2} \Big\rceil \quad \text{para } n \geq 2, \tag{7}$$

que, na forma padrão do Teorema Mestre (desprezando pisos e tetos), se escreve

$$T(n) = 2\,T\Big(\frac{n}{2}\Big) + \frac{n^2}{4}. \tag{8}$$

4.3.3 Valores de a, b e $f(n)$, e cálculo de $n^{\log_b a}$

$$a = 2, \qquad b = 2, \qquad f(n) = \frac{n^2}{4} = \Theta(n^2).$$

$$n^{\log_b a} = n^{\log_2 2} = n^1 = n.$$

4.3.4 Enquadramento e justificativa

Compara-se $f(n) = \Theta(n^2)$ com $n^{\log_2 2} = n$. Como $2 > 1$, a função f é polinomialmente *maior*, o que aponta para o **Caso 3**. A verificação formal tem duas partes.

(i) Crescimento polinomialmente maior. É preciso exibir $\varepsilon > 0$ com $f(n) = \Omega\big(n^{1+\varepsilon}\big)$. Tomando $\varepsilon = 1$, tem-se $n^{1+\varepsilon} = n^2$ e, de fato, $f(n) = n^2/4 \geq \frac{1}{4}n^2$ para todo $n \geq 1$, ou seja, $f(n) = \Omega(n^2)$.

(ii) Condição de regularidade. É preciso exibir $c < 1$ com $a\,f(n/b) \leq c\,f(n)$ para todo n suficientemente grande. Calculando:

$$a\,f\Big(\frac{n}{b}\Big) = 2 \cdot \frac{(n/2)^2}{4} = 2 \cdot \frac{n^2}{16} = \frac{n^2}{8} = \frac{1}{2} \cdot \frac{n^2}{4} = \frac{1}{2}\,f(n).$$

A condição vale com $c = \frac{1}{2} < 1$ para todo $n \geq 1$. Ambas as partes estão satisfeitas, e o Caso 3 se aplica.

Pela árvore de recursão, o custo do nível i é $2^i \cdot \frac{(n/2^i)^2}{4} = \frac{n^2}{4} \cdot \frac{1}{2^i}$: o custo cai pela metade a cada nível descido, e a soma geométrica é dominada pela raiz.

4.3.5 Complexidade assintótica

Pelo Caso 3 do Teorema Mestre,

$$\boxed{T(n) = \Theta\big(f(n)\big) = \Theta\big(n^2\big)\,.}$$

4.3.6 Demonstração por indução

Aqui a demonstração usa *indução forte*, porque a recorrência (7) remete a $\lfloor n/2 \rfloor$ e $\lceil n/2 \rceil$, e não ao valor imediatamente anterior. Em compensação, o resultado obtido vale para *todo* $n \geq 1$, e não apenas para potências de 2.

Proposição 3. *Para todo inteiro $n \geq 1$, a solução da recorrência (7) é*

$$T(n) = \frac{n(n-1)}{2} = \binom{n}{2}.$$

Demonstração. Por indução forte sobre n.

Caso base ($n = 1$). A recorrência dá $T(1) = 0$, e a fórmula dá $\frac{1\cdot 0}{2} = 0$. Os dois valores coincidem.

Hipótese de indução forte. Seja $n \geq 2$ e suponha que, para *todo* j com $1 \leq j < n$, valha $T(j) = \frac{j(j-1)}{2}$.

Passo indutivo. Sejam $p = \lfloor n/2 \rfloor$ e $q = \lceil n/2 \rceil$, de modo que $p + q = n$. Como $n \geq 2$, tem-se $1 \leq p \leq q < n$, e a hipótese de indução forte se aplica tanto a p quanto a q. Pela recorrência (7),

$$\begin{aligned} T(n) &= T(p) + T(q) + p\,q \\ &= \frac{p(p-1)}{2} + \frac{q(q-1)}{2} + pq \\ &= \frac{p^2 - p + q^2 - q + 2pq}{2} \\ &= \frac{(p^2 + 2pq + q^2) - (p+q)}{2} \\ &= \frac{(p+q)^2 - (p+q)}{2} \\ &= \frac{n^2 - n}{2} = \frac{n(n-1)}{2}, \end{aligned}$$

que é precisamente a fórmula avaliada em n.

Pelo princípio da indução forte, a fórmula vale para todo $n \geq 1$. □

Interpretação combinatória. O resultado $T(n) = \binom{n}{2}$ tem uma leitura direta: cada par não ordenado $\{i, j\}$ de posições do vetor é comparado *exatamente uma vez*, no único nível da recursão em que i e j são separados em metades diferentes. O algoritmo, portanto, faz o mesmo trabalho que a força bruta, apenas organizado de forma recursiva — o que confirma, de modo concreto, a leitura do Caso 3: toda a recursão não compra nenhuma economia, pois o custo está na combinação.

Coincidência com o Teorema Mestre. Expandindo,

$$T(n) = \frac{n^2}{2} - \frac{n}{2}, \tag{9}$$

cujo termo dominante é $n^2/2$; logo $T(n) = \Theta(n^2)$, exatamente o resultado obtido pelo Teorema Mestre.

5 Resultados Experimentais e Comparação com a Teoria

Todos os experimentos foram registrados no arquivo `resultados/resultados.csv`, com uma linha por execução. As tabelas desta seção foram geradas automaticamente a partir desse arquivo. A coluna *Previsto* traz o valor da forma fechada demonstrada na Seção 4, e a coluna *Dif.* traz a diferença entre o valor medido e o previsto.

5.1 Problema 1 — Caso 1: Karatsuba

5.1.1 Dados coletados

Tabela 2: *Problema 1: operações medidas e previstas por* $(21 \cdot 3^k - 24 \cdot 2^k + 5)/2$*, com* $n = 2^k$.

n	Operações	Previsto	Dif.	$T(2n)/T(n)$	Tempo (μs)
2	10	10	0	—	0,10
4	49	49	0	4,9000	0,28
8	190	190	0	3,8776	0,91
16	661	661	0	3,4789	2,81
32	2.170	2.170	0	3,2829	8,76
64	6.889	6.889	0	3,1747	26,82
128	21.430	21.430	0	3,1108	82,21
256	65.821	65.821	0	3,0714	240,00
512	200.530	200.530	0	3,0466	770,28
1.024	607.729	607.729	0	3,0306	2.247,68
2.048	1.835.470	1.835.470	0	3,0202	8.165,74
4.096	5.530.981	5.530.981	0	3,0134	20.226,79

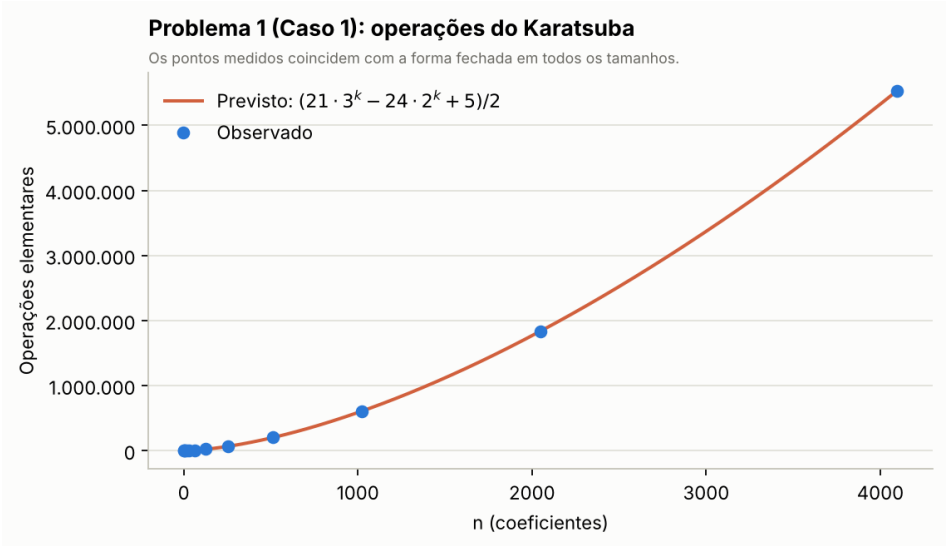


Figura 1: *Problema 1: pontos medidos sobre a curva teórica da forma fechada* (4).

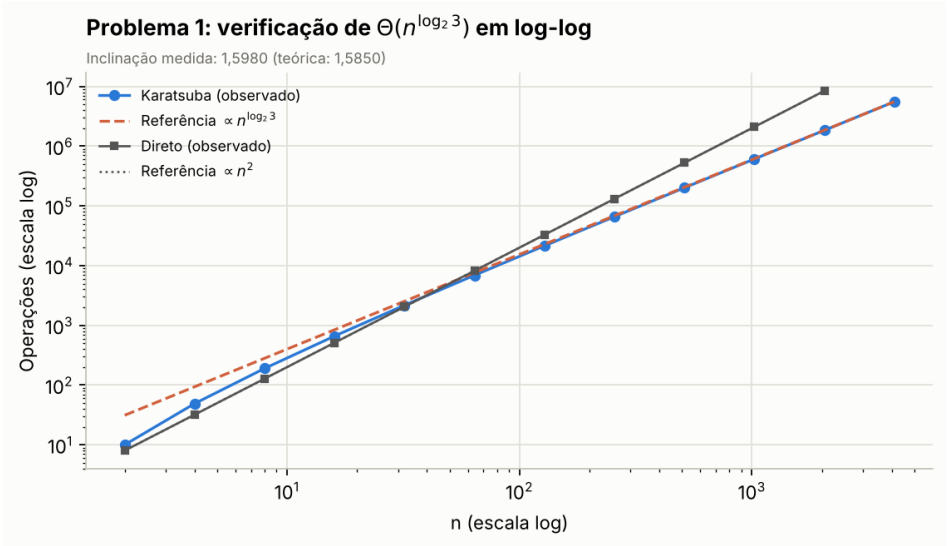


Figura 2: *Problema 1 em escala log-log, com as retas de referência $n^{\log_2 3}$ e n^2. As duas implementações têm inclinações visivelmente distintas.*

5.1.2 Análise e comparação com a teoria

A diferença entre medido e previsto é **zero em todos os doze tamanhos**, incluindo o maior, em que a contagem passa de cinco milhões de operações. A coincidência é exata, e não aproximada, porque o contador mede exatamente a grandeza que a recorrência (3) descreve: cada operação sobre coeficientes.

A classe assintótica se confirma por dois indicadores independentes da constante multiplicativa:

- **Razão de crescimento.** Ao dobrar n, a razão $T(2n)/T(n)$ cai monotonicamente de $4{,}9000$ para $3{,}0134$, convergindo para $a = 3$. O valor nunca é exatamente 3 porque os termos $-12n + 2{,}5$ da forma fechada (4) ainda pesam; sua influência relativa decai à medida que n cresce.
- **Inclinação log-log.** O ajuste por mínimos quadrados sobre os cinco maiores tamanhos dá inclinação $1{,}5980$ para as operações e $1{,}6200$ para os tempos, ambas próximas do expoente teórico $\log_2 3 \approx 1{,}5850$. O desvio residual é consequência dos termos de menor ordem, que puxam a inclinação para cima nessa faixa de n.

5.1.3 Comparação com o método direto

A implementação também executou a multiplicação direta, de custo $2n^2$ operações, para servir de referência experimental ao Caso 1.

Tabela 3: *Problema 1: Karatsuba comparado à multiplicação direta.*

n	Ops. Karatsuba	Ops. direto	Ganho	t Karatsuba (μs)	t direto (μs)	Ganho
2	10	8	0,80	0,10	0,05	0,50
4	49	32	0,65	0,28	0,07	0,26
8	190	128	0,67	0,91	0,20	0,22
16	661	512	0,77	2,81	0,71	0,25
32	2.170	2.048	0,94	8,76	2,77	0,32
64	6.889	8.192	1,19	26,82	10,87	0,41
128	21.430	32.768	1,53	82,21	42,07	0,51
256	65.821	131.072	1,99	240,00	166,07	0,69
512	200.530	524.288	2,61	770,28	704,10	0,91
1.024	607.729	2.097.152	3,45	2.247,68	2.655,53	1,18
2.048	1.835.470	8.388.608	4,57	8.165,74	10.439,14	1,28

O comportamento é o previsto pela teoria assintótica, com uma ressalva prática importante. Em *número de operações*, o Karatsuba é pior que o método direto para $n \leq 32$ e só passa a vencer a partir de $n = 64$; isso ocorre porque a constante multiplicativa de $10{,}5\, n^{\log_2 3}$ é maior que a de $2n^2$, e a vantagem assintótica leva tempo para compensá-la. Em *tempo de execução*, o ponto de virada é ainda mais tardio: o Karatsuba só fica mais rápido a partir de $n = 1024$. A razão é que cada chamada recursiva faz cinco alocações de memória e uma chamada de função, custos reais que o contador de operações não registra. Em $n = 2.048$, o Karatsuba faz $4{,}57$ vezes menos operações, mas é apenas $1{,}28$ vez mais rápido.

Esta é a primeira divergência relevante entre teoria e prática observada no trabalho, e ela não contradiz o Teorema Mestre: o teorema descreve o comportamento *assintótico*, e nada afirma sobre a faixa de n em que as constantes dominam.

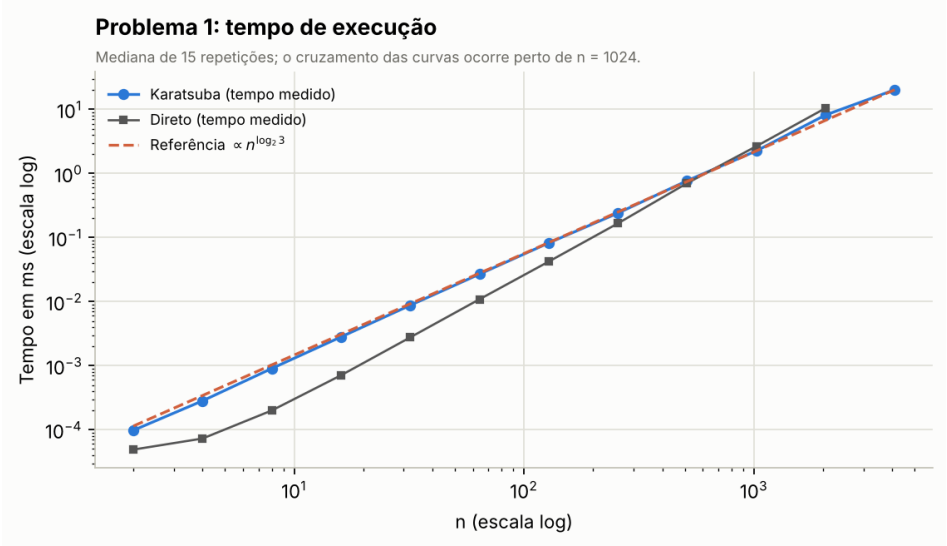


Figura 3: *Problema 1: tempo de execução das duas implementações. O cruzamento das curvas ocorre perto de $n = 1024$, bem depois do cruzamento em número de operações.*

5.2 Problema 2 — Caso 2: MergeSort ternário

5.2.1 Dados coletados

Tabela 4: *Problema 2: operações medidas e previstas por $n \log_3 n$, com $n = 3^k$. A coluna* Prev. *traz a razão prevista exatamente por $3(k+1)/k$.*

k	n	Operações	Previsto	Dif.	$T(3n)/T(n)$	Prev.	Tempo (μs)
1	3	3	3	0	—	—	0,05
2	9	18	18	0	6,0000	6,0000	0,11
3	27	81	81	0	4,5000	4,5000	0,37
4	81	324	324	0	4,0000	4,0000	1,34
5	243	1.215	1.215	0	3,7500	3,7500	4,47
6	729	4.374	4.374	0	3,6000	3,6000	15,31
7	2.187	15.309	15.309	0	3,5000	3,5000	97,88
8	6.561	52.488	52.488	0	3,4286	3,4286	410,99
9	19.683	177.147	177.147	0	3,3750	3,3750	1.432,88
10	59.049	590.490	590.490	0	3,3333	3,3333	4.885,29
11	177.147	1.948.617	1.948.617	0	3,3000	3,3000	16.352,71
12	531.441	6.377.292	6.377.292	0	3,2727	3,2727	49.505,57

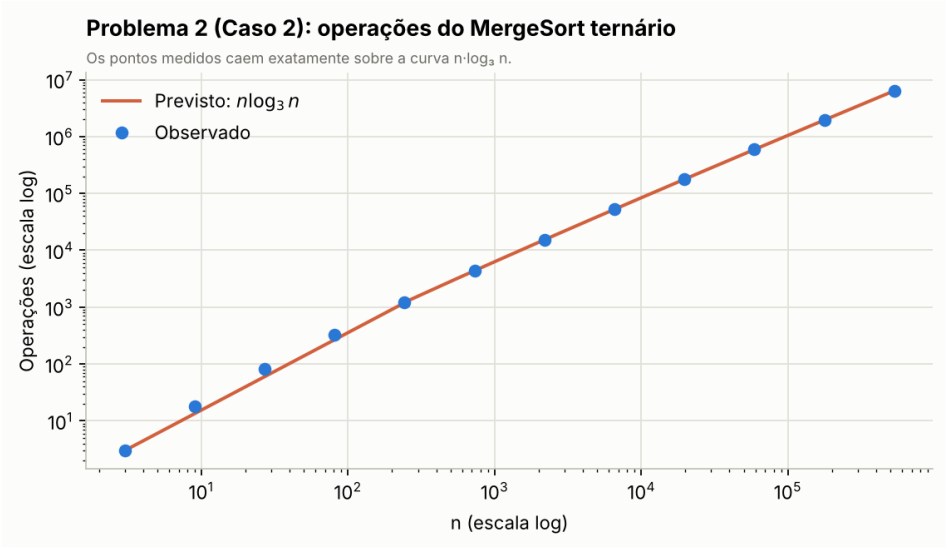


Figura 4: *Problema 2: operações medidas sobre a curva $n \log_3 n$, em escala log-log.*

5.2.2 Análise e comparação com a teoria

Também aqui a diferença entre medido e previsto é **zero em todos os doze tamanhos**. A correção da ordenação foi verificada a cada execução, percorrendo o vetor resultante.

O Caso 2 tem uma assinatura experimental própria, que o distingue dos outros dois: a razão de crescimento *não* converge para $a = 3$ em um valor fixo, mas se aproxima dele por valores maiores, segundo uma lei exata. De fato, para $n = 3^k$,

$$\frac{T(3n)}{T(n)} = \frac{(k+1)\,3^{k+1}}{k\,3^k} = 3 \cdot \frac{k+1}{k} = 3 + \frac{3}{k}.$$

A coluna *Prev.* da Tabela 4 traz esse valor, e a concordância com a razão medida é exata em todas as onze transições, até a quarta casa decimal. O excesso $3/k$ sobre o limite 3 é justamente a manifestação do fator logarítmico do Caso 2: ele decai como $1/\lg n$, e não como uma potência de n.

Outra forma de ver o mesmo fenômeno é isolar o fator logarítmico. Dividindo as operações medidas por n, resta exatamente $\log_3 n$, ou seja, os valores inteiros $1, 2, 3, \ldots, 12$ — uma reta perfeita quando n é plotado em escala logarítmica.

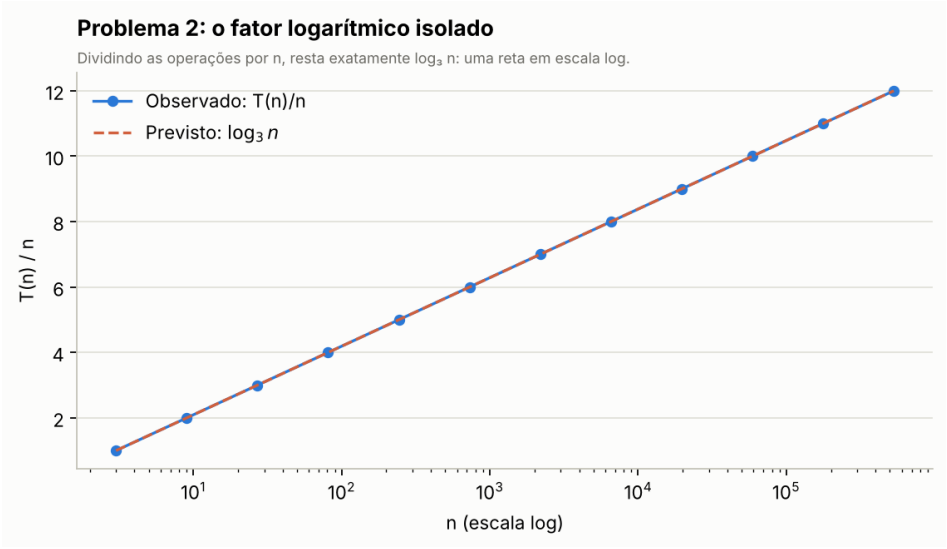


Figura 5: *Problema 2: dividindo as operações por* n*, resta exatamente* $\log_3 n$*. A razão medida vale* $1{,}000000$ *em todos os doze pontos.*

A inclinação log-log sobre os cinco maiores tamanhos é $1{,}0921$ para as operações e $1{,}0939$ para os tempos. Esses valores *não* devem ser comparados a 1, e sim à inclinação local da função $n \lg n$, que é

$$\frac{d \ln(n \ln n)}{d \ln n} = 1 + \frac{1}{\ln n}.$$

Para n entre 19.683 e 531.441, isso dá entre $1{,}076$ e $1{,}101$, faixa que contém os dois valores medidos. A curva medida, portanto, é levemente supralinear, exatamente como prevê $\Theta(n \lg n)$, e claramente distinta de uma reta de inclinação 2, que seria o comportamento quadrático.

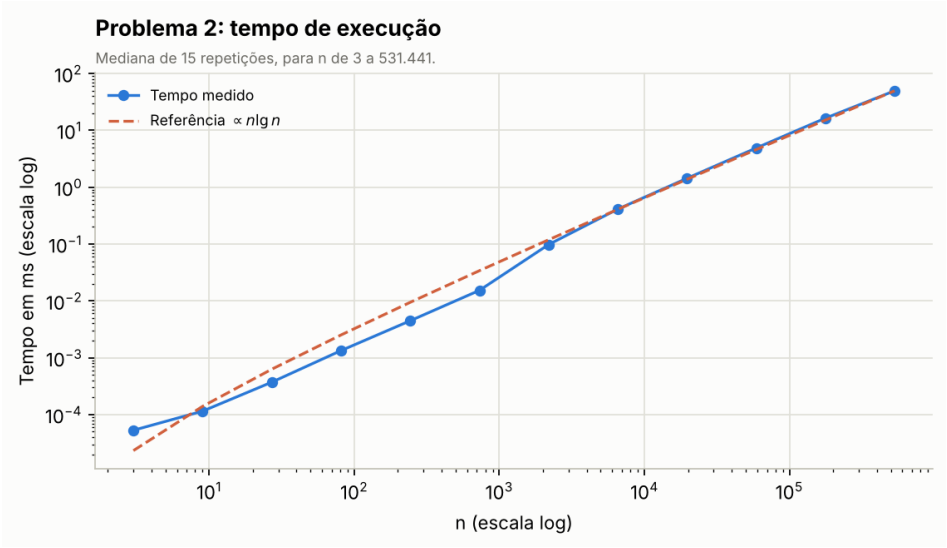


Figura 6: *Problema 2: tempo de execução medido, com reta de referência proporcional a* $n \lg n$*.*

5.3 Problema 3 — Caso 3: par mais próximo

5.3.1 Dados coletados

Tabela 5: *Problema 3: comparações medidas e previstas por* $n(n-1)/2$.

n	Comparações	Previsto	Dif.	$T(2n)/T(n)$	Tempo (μs)
2	1	1	0	—	0,04
4	6	6	0	6,0000	0,04
8	28	28	0	4,6667	0,08
16	120	120	0	4,2857	0,18
32	496	496	0	4,1333	0,56
64	2.016	2.016	0	4,0645	1,99
128	8.128	8.128	0	4,0317	7,62
256	32.640	32.640	0	4,0157	29,65
512	130.816	130.816	0	4,0078	119,98
1.024	523.776	523.776	0	4,0039	481,57
2.048	2.096.128	2.096.128	0	4,0020	1.911,85
4.096	8.386.560	8.386.560	0	4,0010	8.183,06
8.192	33.550.336	33.550.336	0	4,0005	32.548,37

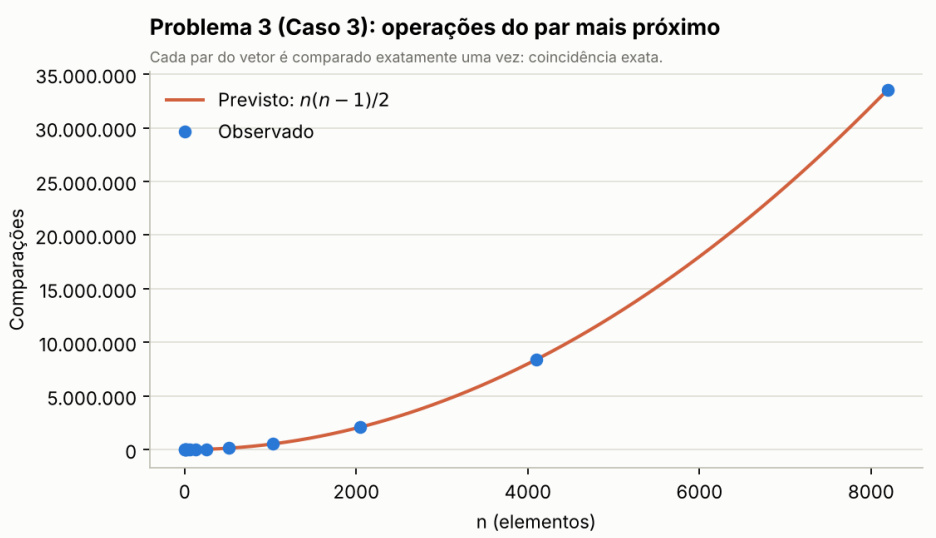


Figura 7: *Problema 3: comparações medidas sobre a curva* $n(n-1)/2$.

5.3.2 Análise e comparação com a teoria

A diferença entre medido e previsto é **zero em todos os treze tamanhos**. O maior caso, $n = 8.192$, produz $33.550.336$ comparações, valor que coincide dígito a dígito com $\binom{8192}{2}$.

A razão de crescimento converge para $a \cdot b^{\varepsilon} = 4$, isto é, para o quadrado do fator de divisão, e a convergência é rápida: a razão vale $4{,}0005$ já em $n = 8.192$. Isso é consistente com a forma fechada (9), em que o termo de menor ordem é apenas $-n/2$, linear, e portanto desprezível diante de $n^2/2$ bem mais cedo do que no Problema 1. A inclinação log-log sobre os cinco maiores tamanhos é $2{,}0006$ para as operações e $2{,}0254$ para os tempos, contra o valor teórico 2.

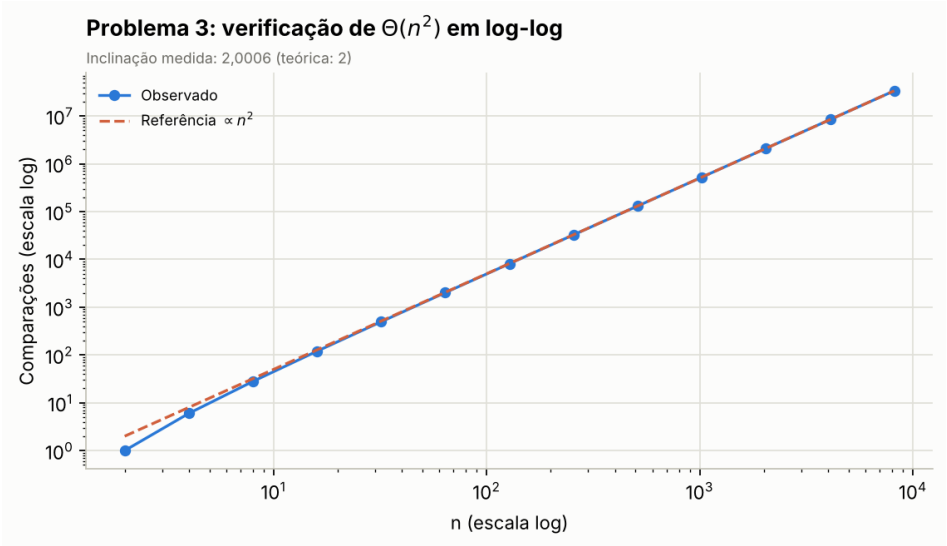


Figura 8: *Problema 3: verificação de* $\Theta(n^2)$ *em escala log-log.*

Vale destacar o que o Caso 3 significa concretamente neste algoritmo. O custo das chamadas recursivas é assintoticamente irrelevante: toda a complexidade está na etapa de combinação. Isso é confirmado pela interpretação combinatória da Proposição 3 — cada par é comparado exatamente uma vez, e o algoritmo recursivo faz exatamente o mesmo número de comparações que uma varredura por força bruta com dois laços encaixados. A divisão e conquista, neste caso, não produz ganho algum; ela apenas reorganiza o mesmo trabalho.

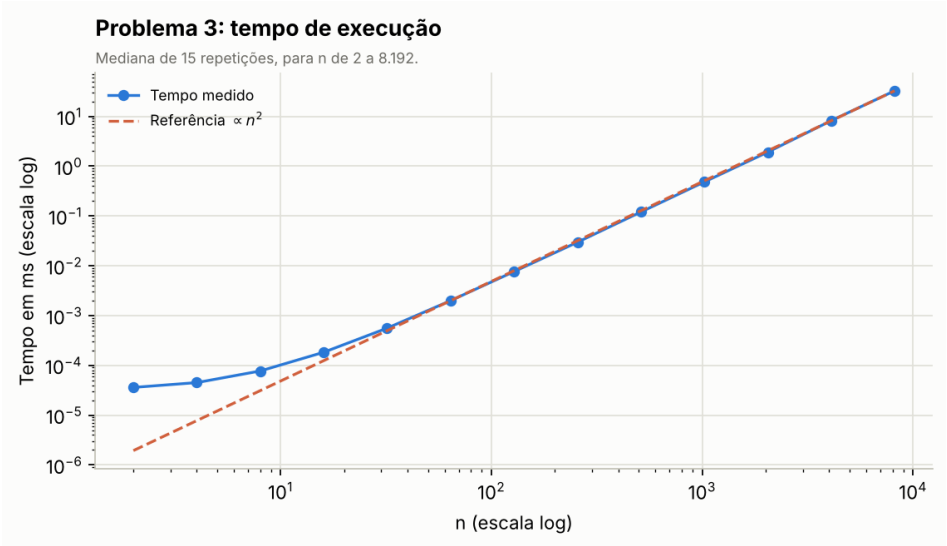


Figura 9: *Problema 3: tempo de execução medido, com reta de referência proporcional a* n^2.

5.4 Comparação entre os três casos

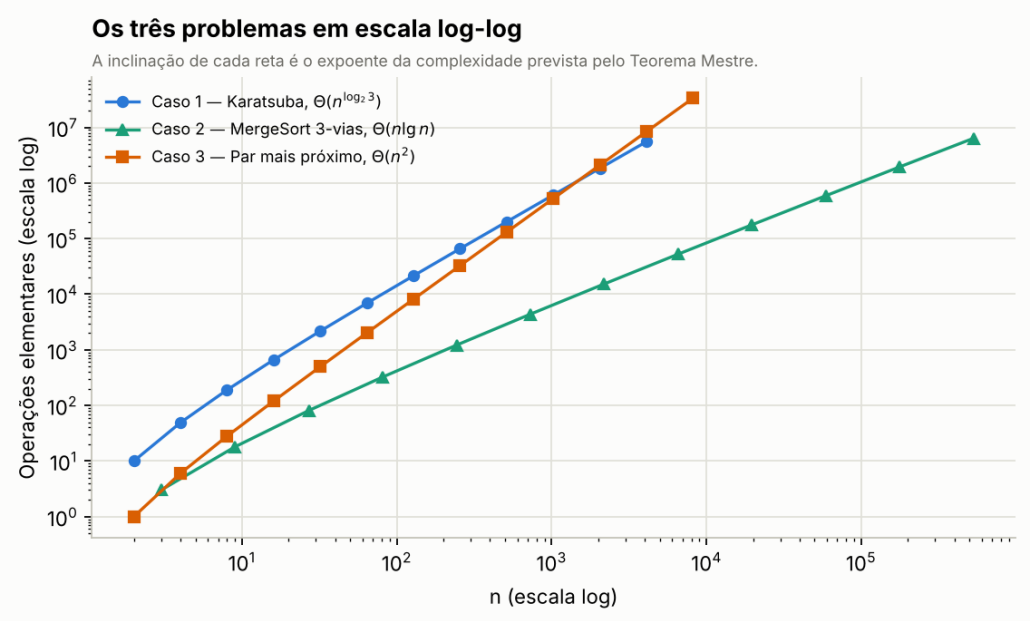


Figura 10: *Os três problemas em escala log-log. A inclinação de cada reta corresponde ao expoente previsto pelo Teorema Mestre:* $\approx 1{,}585$ *no Caso 1, levemente acima de 1 no Caso 2 e 2 no Caso 3.*

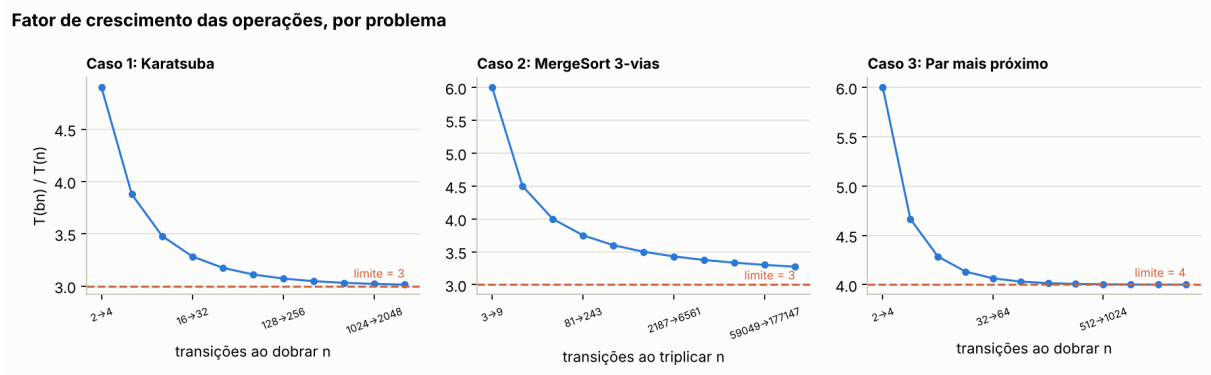


Figura 11: *Fator de crescimento das operações em cada problema. Em todos, a razão converge monotonicamente para o limite previsto, mas com velocidades bem diferentes.*

A comparação entre as três curvas de razão revela uma diferença qualitativa entre os casos que o enunciado do Teorema Mestre já antecipa. Nos Casos 1 e 3, o limite da razão é atingido por convergência rápida, porque os termos de menor ordem são polinomialmente menores que o dominante. No Caso 2, a convergência é lenta: o excesso sobre o limite decai como $3/k = 3/\log_3 n$, de modo que mesmo em $n = 531.441$ a razão ainda vale 3,2727, longe de 3. Um experimento que olhasse apenas para a razão, sem conhecer a lei $3 + 3/k$, poderia concluir erroneamente que o limite não é 3.

5.5 Custo real de cada operação elementar

A Tabela 6 traz o tempo medido dividido pelo número de operações contadas, para o menor tamanho de cada problema e para a faixa dos seis maiores.

Tabela 6: *Nanossegundos por operação elementar: no menor tamanho e na faixa dos seis maiores.*

Problema	Menor n	Mínimo (maiores)	Máximo (maiores)
Problema 1 — Karatsuba	9,8	3,65	4,45
Problema 2 — MergeSort ternário	17,7	6,39	8,39
Problema 3 — Par mais próximo	36,0	0,91	0,98

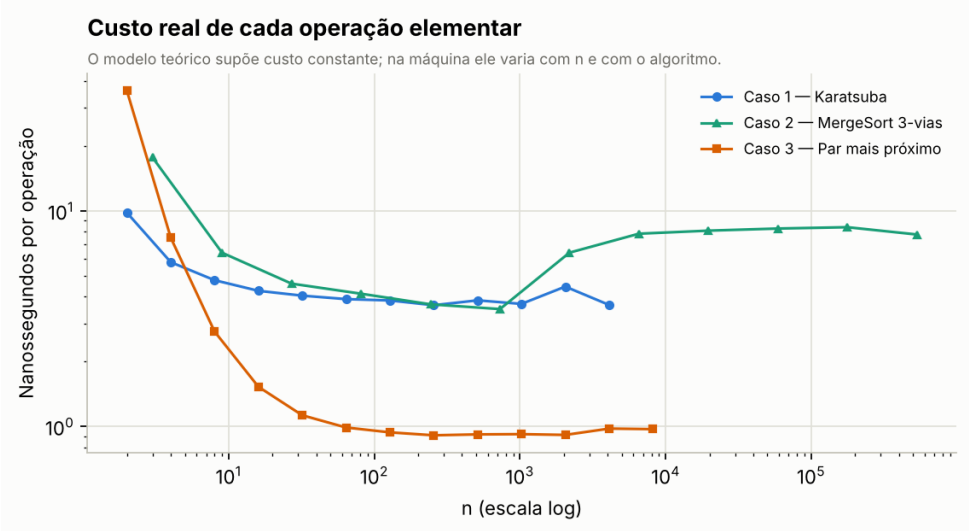


Figura 12: *Custo real de cada operação elementar em função de n. O modelo teórico supõe esse custo constante; na máquina real ele varia tanto com o algoritmo quanto com o tamanho da entrada.*

Os dados mostram que a hipótese de custo uniforme, embutida no Teorema Mestre, é uma simplificação. Três observações:

- **O custo por operação difere entre algoritmos.** Uma comparação no Problema 3 custa cerca de $0{,}95$ ns, enquanto uma operação do Problema 1 custa cerca de 4 ns e uma do Problema 2 chega a 8 ns. O laço do Problema 3 é um acesso sequencial simples, favorável à *cache* e passível de vetorização pelo compilador; o do Problema 1 envolve recursão e alocações; o do Problema 2 envolve três ponteiros simultâneos e uma cópia de memória adicional.
- **O custo por operação cresce com n no Problema 2.** Ele sobe de $3{,}5$ ns em $n = 729$ para $8{,}4$ ns em $n = 177.147$. A causa é a hierarquia de memória: quando o vetor e seu auxiliar deixam de caber na *cache*, cada acesso passa a custar mais. Esse efeito explica por que a inclinação log-log dos tempos ($1{,}0939$) é ligeiramente maior que a das operações ($1{,}0921$).
- **O custo por operação é alto nas entradas pequenas.** Nos menores tamanhos, o valor é de uma a duas ordens de grandeza maior, porque o tempo total é dominado por custos fixos: a chamada ao relógio, a entrada e a saída da função, a alocação inicial. Esses pontos foram, por isso, excluídos dos ajustes de inclinação.

5.6 Síntese das convergências e divergências

Tabela 7: *Resumo da comparação entre teoria e experimento.*

	Problema 1	Problema 2	Problema 3
Caso do Teorema Mestre	1	2	3
Complexidade prevista	$\Theta(n^{\log_2 3})$	$\Theta(n \lg n)$	$\Theta(n^2)$
Execuções registradas	12	12	13
Diferença máxima (operações)	0	0	0
Razão medida (maior n)	3,0134	3,2727	4,0005
Razão prevista	$\to 3$	$3 + 3/k$ (exata)	$\to 4$
Inclinação log-log, operações	1,5980	1,0921	2,0006
Inclinação log-log, tempo	1,6200	1,0939	2,0254
Expoente teórico	1,5850	$1 + 1/\ln n$	2,0000

Convergências. Nos três problemas, a contagem de operações coincidiu exatamente com a forma fechada demonstrada por indução, sem nenhuma diferença, em 37 execuções. As taxas de crescimento e as inclinações log-log convergiram para os valores previstos pelo Teorema Mestre, e no Problema 2 a razão medida reproduziu a lei exata $3 + 3/k$ em todas as transições.

Divergências. Elas aparecem apenas quando se passa das operações para o tempo, e têm três causas identificadas, nenhuma delas um erro do modelo teórico: (i) *constantes multiplicativas*, responsáveis pelo cruzamento tardio entre Karatsuba e método direto; (ii) *operações não contabilizadas*, como alocações e chamadas de função, que pesam no tempo e não no contador; e (iii) *hierarquia de memória*, que faz o custo por operação crescer com n e inflaciona levemente as inclinações medidas sobre os tempos. Todas são consequências do fato de que o Teorema Mestre descreve o número de operações em um modelo de custo uniforme, e não o tempo em uma máquina real.

6 Conclusão

6.1 Resumo dos resultados

Foram escolhidos, implementados e analisados três algoritmos recursivos, um para cada caso do Teorema Mestre:

- **Problema 1, Caso 1:** multiplicação de polinômios por Karatsuba, $T(n) = 3T(n/2)+6n-5$, com $a = 3$, $b = 2$, $f(n) = \Theta(n)$ e $n^{\log_2 3} \approx n^{1{,}5850}$. Como $f(n) = O(n^{\log_2 3-\varepsilon})$ com $\varepsilon = \log_2 3 - 1$, aplica-se o Caso 1 e $T(n) = \Theta(n^{\log_2 3})$. A indução estabeleceu $T(2^k) = (21 \cdot 3^k - 24 \cdot 2^k + 5)/2$.
- **Problema 2, Caso 2:** MergeSort ternário, $T(n) = 3T(n/3) + n$, com $a = 3$, $b = 3$, $f(n) = \Theta(n)$ e $n^{\log_3 3} = n$. Como $f(n) = \Theta(n^{\log_3 3})$, aplica-se o Caso 2 e $T(n) = \Theta(n \lg n)$. A indução estabeleceu $T(3^k) = k \cdot 3^k$, isto é, $T(n) = n \log_3 n$.
- **Problema 3, Caso 3:** par mais próximo em 1D com combinação por força bruta, $T(n) = 2T(n/2) + n^2/4$, com $a = 2$, $b = 2$, $f(n) = \Theta(n^2)$ e $n^{\log_2 2} = n$. Como $f(n) = \Omega(n^{1+\varepsilon})$ com $\varepsilon = 1$ e a condição de regularidade vale com $c = 1/2$, aplica-se o Caso 3 e $T(n) = \Theta(n^2)$. A indução forte estabeleceu $T(n) = n(n-1)/2$, válida para todo $n \geq 1$.

As três formas fechadas demonstradas coincidem, em termo dominante, com a complexidade prevista pelo Teorema Mestre, o que cumpre a exigência de que os dois métodos levem ao mesmo resultado.

6.2 Síntese da comparação entre teoria e prática

A validação experimental foi conduzida em 37 execuções, com n variando de 2 a 531.441. Em *todas* elas, a diferença entre o número de operações medido e o previsto pela forma fechada foi exatamente zero. As assinaturas assintóticas também se confirmaram: a razão de crescimento convergiu para 3, para $3 + 3/k$ e para 4, respectivamente, e as inclinações log-log ficaram em $1{,}5980$, $1{,}0921$ e $2{,}0006$, contra os valores teóricos $1{,}5850$, $1 + 1/\ln n$ e 2.

As divergências se concentraram nos tempos de execução e foram todas explicadas por fatores externos ao modelo: constantes multiplicativas, operações não contabilizadas pelo contador e efeitos da hierarquia de memória. Nenhuma delas contradiz o Teorema Mestre; elas delimitam o que o teorema afirma e o que não afirma.

6.3 Principais observações

1. **Cada caso tem uma assinatura experimental distinta.** A razão $T(bn)/T(n)$ converge para a no Caso 1, para a *lentamente* e com excesso $\Theta(1/\lg n)$ no Caso 2, e para $a \cdot b^{\varepsilon}$ no Caso 3. Observar a razão, e não apenas a curva, foi o indicador mais informativo de todo o trabalho.

2. **A superioridade assintótica demora a se manifestar.** O Karatsuba é $\Theta(n^{1{,}585})$ contra $\Theta(n^2)$ do método direto, mas só passa a fazer menos operações a partir de $n = 64$ e só fica mais rápido a partir de $n = 1024$. Para entradas pequenas, a constante multiplicativa importa mais que o expoente.

3. **No Caso 3, a recursão pode não trazer ganho algum.** A Proposição 3 mostra que o algoritmo do Problema 3 faz exatamente $\binom{n}{2}$ comparações, o mesmo que a força bruta. O Caso 3 do Teorema Mestre é, nesse sentido, um alerta de projeto: quando o custo da combinação domina, dividir e conquistar não compra nada.

4. **Contar operações é mais informativo que cronometrar.** Foi a contagem, e não o tempo, que permitiu verificar as formas fechadas dígito a dígito. O tempo serviu a outro propósito: mostrar onde o modelo de custo uniforme se afasta da máquina real.

5. **A hipótese de custo uniforme é a maior simplificação do modelo.** O custo real de uma operação variou de $0{,}91$ ns a $8{,}4$ ns entre os algoritmos, e cresceu com n dentro de um mesmo algoritmo.

6.4 Limitações encontradas

Além das limitações metodológicas já discutidas, duas restrições do próprio Teorema Mestre ficaram evidentes. A primeira é que o teorema fornece apenas a classe assintótica: sozinho, ele não permitiria prever nenhum dos valores das tabelas, nem responder em que n o Karatsuba supera o método direto. As formas fechadas obtidas por indução foram necessárias para isso. A segunda é que as formas fechadas exatas dos Problemas 1 e 2 valem apenas para n potência de b; para n arbitrário, a recorrência envolve pisos e tetos, e o que permanece válido é a classe assintótica. O Problema 3 é a exceção, pois sua forma fechada foi demonstrada por indução forte para todo n.

Por fim, a faixa de tamanhos foi limitada pelo custo computacional, sobretudo no Problema 3, cujo custo quadrático impediu ir além de $n = 8.192$ em tempo razoável.

6.5 Considerações adicionais

Dois pontos merecem registro e não couberam na seção de comparação.

O primeiro é que a escolha da *operação elementar* a contar não é neutra, e foi decisiva para a qualidade dos resultados. Nos três problemas, ela foi definida de modo a não depender dos valores da entrada, apenas do seu tamanho. Se, no Problema 2, tivéssemos contado *comparações* entre elementos em vez

de *posicionamentos*, a contagem passaria a depender da ordem dos dados, e a coincidência exata com $n \log_3 n$ daria lugar a uma faixa de valores entre um melhor e um pior caso. A exatidão observada é, em parte, consequência dessa escolha de instrumentação.

O segundo é que os três problemas ilustram usos muito diferentes da divisão e conquista. No Caso 1, ela é a fonte do ganho: reduzir de quatro para três subproblemas é exatamente o que baixa o expoente de 2 para $\log_2 3$. No Caso 2, ela equilibra o trabalho entre os níveis, e o preço do balanceamento é o fator logarítmico. No Caso 3, ela é inócua. A posição de um algoritmo entre os três casos é, portanto, menos uma classificação matemática do que um diagnóstico sobre onde está o seu gargalo — e, consequentemente, sobre onde faz sentido tentar otimizá-lo.

A Código-fonte

Todos os arquivos abaixo foram compilados com

```
gcc -O2 -Wall -Wextra -std=c99 -o experimentos src/main.c src/comum.c \
    src/problema1.c src/problema2.c src/problema3.c -lm
```

e executados com `./experimentos resultados/resultados.csv 15`.

src/comum.h

```c
/* comum.h  contador de operações, medição de tempo e gravação do CSV. */
#ifndef COMUM_H
#define COMUM_H

#include <stddef.h>

/* Contador global de operações elementares. Cada algoritmo o incrementa
   exatamente uma vez por operação elementar, conforme definido no relatório. */
extern unsigned long long operacoes;

/* Relógio monotônico de alta resolução (Windows e Linux/macOS). */
double agora_segundos(void);

/* Mediana de um vetor de tempos (ordena uma cópia). */
double mediana(double *v, int n);

/* Acrescenta uma linha ao arquivo CSV de resultados, criando o cabeçalho
   se o arquivo ainda não existir. Todos os experimentos dos três problemas
   são gravados neste mesmo arquivo. */
void registrar(const char *caminho, int problema, const char *algoritmo,
               const char *recorrencia, size_t n, unsigned long long ops,
               unsigned long long previsto, double tempo_s, int repeticoes);

#endif
```

src/comum.c

```c
/* comum.c  implementação dos utilitários comuns aos três problemas. */

#define _POSIX_C_SOURCE 199309L

#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include "comum.h"

unsigned long long operacoes = 0;

#if defined(_WIN32) || defined(_WIN64)
#include <windows.h>
double agora_segundos(void) {
```

```
    LARGE_INTEGER freq, cont;
    QueryPerformanceFrequency(&freq);
    QueryPerformanceCounter(&cont);
    return (double)cont.QuadPart / (double)freq.QuadPart;
}
#else
double agora_segundos(void) {
    struct timespec ts;
    clock_gettime(CLOCK_MONOTONIC, &ts);
    return (double)ts.tv_sec + (double)ts.tv_nsec / 1e9;
}
#endif

static int cmp_double(const void *a, const void *b) {
    double x = *(const double *)a, y = *(const double *)b;
    return (x > y) - (x < y);
}

double mediana(double *v, int n) {
    qsort(v, (size_t)n, sizeof(double), cmp_double);
    return (n % 2) ? v[n / 2] : (v[n / 2 - 1] + v[n / 2]) / 2.0;
}

void registrar(const char *caminho, int problema, const char *algoritmo,
               const char *recorrencia, size_t n, unsigned long long ops,
               unsigned long long previsto, double tempo_s, int repeticoes) {
    FILE *fp = fopen(caminho, "a+");
    if (!fp) { perror("ERRO ao abrir o arquivo de resultados"); return; }
    fseek(fp, 0, SEEK_END);
    if (ftell(fp) == 0)
        fprintf(fp, "problema,algoritmo,recorrencia,n,operacoes,previsto,"
                    "diferenca,tempo_mediano_s,repeticoes\n");
    fprintf(fp, "%d,%s,\"%s\",%zu,%llu,%llu,%lld,%.9f,%d\n",
            problema, algoritmo, recorrencia, n, ops, previsto,
            (long long)ops - (long long)previsto, tempo_s, repeticoes);
    fclose(fp);
}
```

src/problema1.c — Caso 1

```
/* problema1.c  Caso 1 do Teorema Mestre.
 *
 * Multiplicação de polinômios pelo algoritmo de Karatsuba.
 *
 * Dados dois polinômios de n coeficientes, P e Q, o algoritmo os divide ao
 * meio, P = P0 + P1*x^m e Q = Q0 + Q1*x^m com m = n/2, e obtém o produto
 * com apenas TRÊS multiplicações de tamanho m, em vez das quatro do método
 * direto:
 *     z0 = P0*Q0
 *     z2 = P1*Q1
 *     z1 = (P0+P1)*(Q0+Q1) - z0 - z2
 *     PQ = z0 + z1*x^m + z2*x^(2m)
 *
 * Recorrência:  T(n) = 3*T(n/2) + 6n - 5,  T(1) = 1.
 * Operação elementar contada: cada soma, subtração ou multiplicação de
 * coeficientes. Cópias de memória não são contadas.
 */

#include <stdlib.h>
#include <string.h>
#include "comum.h"

/* Multiplicação direta, O(n^2), usada como referência experimental. */
void multiplicar_direto(const long long *p, const long long *q, size_t n,
                        long long *res) {
    size_t i, j;
    for (i = 0; i < 2 * n - 1; i++) res[i] = 0;
    for (i = 0; i < n; i++)
        for (j = 0; j < n; j++) {
            operacoes += 2;             /* uma multiplicação e uma soma */
            res[i + j] += p[i] * q[j];
```

```
        }
}

/* Karatsuba. Requer n potência de 2. O vetor res recebe 2n-1 coeficientes. */
void karatsuba(const long long *p, const long long *q, size_t n,
               long long *res) {
    size_t m, i, tam;
    long long *soma_p, *soma_q, *z0, *z1, *z2;

    if (n == 1) {                       /* caso base da recursão */
        operacoes += 1;
        res[0] = p[0] * q[0];
        return;
    }

    m = n / 2;
    tam = 2 * m - 1;                    /* coeficientes de cada produto parcial */

    soma_p = malloc(m * sizeof(long long));
    soma_q = malloc(m * sizeof(long long));
    z0 = malloc(tam * sizeof(long long));
    z1 = malloc(tam * sizeof(long long));
    z2 = malloc(tam * sizeof(long long));
    if (!soma_p || !soma_q || !z0 || !z1 || !z2) exit(EXIT_FAILURE);

    /* P0+P1 e Q0+Q1: 2m operações. */
    for (i = 0; i < m; i++) { operacoes++; soma_p[i] = p[i] + p[i + m]; }
    for (i = 0; i < m; i++) { operacoes++; soma_q[i] = q[i] + q[i + m]; }

    /* As três chamadas recursivas, todas de tamanho n/2. */
    karatsuba(p, q, m, z0);
    karatsuba(p + m, q + m, m, z2);
    karatsuba(soma_p, soma_q, m, z1);

    /* z1 = (P0+P1)(Q0+Q1) - z0 - z2: 2*(2m-1) operações. */
    for (i = 0; i < tam; i++) { operacoes++; z1[i] -= z0[i]; }
    for (i = 0; i < tam; i++) { operacoes++; z1[i] -= z2[i]; }

    /* Combinação: res = z0 + z1*x^m + z2*x^(2m), 3*(2m-1) operações. */
    for (i = 0; i < 2 * n - 1; i++) res[i] = 0;
    for (i = 0; i < tam; i++) { operacoes++; res[i] += z0[i]; }
    for (i = 0; i < tam; i++) { operacoes++; res[i + m] += z1[i]; }
    for (i = 0; i < tam; i++) { operacoes++; res[i + 2 * m] += z2[i]; }

    free(soma_p); free(soma_q); free(z0); free(z1); free(z2);
}
```

src/problema2.c — Caso 2

```
/* problema2.c  Caso 2 do Teorema Mestre.
 *
 * MergeSort ternário: o vetor é dividido em TRÊS partes de tamanho n/3,
 * cada uma ordenada recursivamente, e as três partes ordenadas são
 * intercaladas em uma única passagem.
 *
 * Recorrência:  T(n) = 3*T(n/3) + n,  T(1) = T(0) = 0.
 * Operação elementar contada: cada posicionamento de um elemento no vetor
 * de saída durante a intercalação. Como a intercalação de três blocos que
 * somam n elementos posiciona exatamente n elementos, a contagem independe
 * dos valores de entrada.
 */

#include <stdlib.h>
#include <string.h>
#include "comum.h"

/* Intercala os três blocos ordenados [ini,c1), [c1,c2) e [c2,fim). */
static void intercalar3(int *v, int *aux, size_t ini, size_t c1, size_t c2,
                        size_t fim) {
    size_t i = ini, j = c1, k = c2, t = ini;

```

```
    while (i < c1 || j < c2 || k < fim) {
        int menor_valor = 0;
        int origem = -1;                /* 0, 1 ou 2: bloco do menor elemento */

        if (i < c1)                       { menor_valor = v[i]; origem = 0; }
        if (j < c2 && (origem < 0 || v[j] < menor_valor))
                                          { menor_valor = v[j]; origem = 1; }
        if (k < fim && (origem < 0 || v[k] < menor_valor))
                                          { menor_valor = v[k]; origem = 2; }

        operacoes++;                    /* um elemento posicionado na saída */
        aux[t++] = menor_valor;
        if (origem == 0) i++;
        else if (origem == 1) j++;
        else k++;
    }
    memcpy(v + ini, aux + ini, (fim - ini) * sizeof(int));
}

/* Ordena recursivamente o intervalo [ini, fim). */
static void mergesort3_aux(int *v, int *aux, size_t ini, size_t fim) {
    size_t tamanho = fim - ini, c1, c2;

    if (tamanho < 2) return;            /* caso base: 0 ou 1 elemento */

    c1 = ini + tamanho / 3;
    c2 = ini + (2 * tamanho) / 3;

    mergesort3_aux(v, aux, ini, c1);
    mergesort3_aux(v, aux, c1, c2);
    mergesort3_aux(v, aux, c2, fim);

    intercalar3(v, aux, ini, c1, c2, fim);
}

void mergesort3(int *v, size_t n) {
    int *aux;
    if (n < 2) return;
    aux = malloc(n * sizeof(int));
    if (!aux) exit(EXIT_FAILURE);
    mergesort3_aux(v, aux, 0, n);
    free(aux);
}
```

src/problema3.c — Caso 3

```
/* problema3.c  Caso 3 do Teorema Mestre.
 *
 * Par mais próximo em uma dimensão, por divisão e conquista com etapa de
 * combinação por força bruta: o vetor é dividido ao meio, a menor distância
 * de cada metade é obtida recursivamente, e a etapa de combinação compara
 * TODOS os pares formados por um elemento de cada metade.
 *
 * Recorrência:  T(n) = T(ceil(n/2)) + T(floor(n/2)) + ceil(n/2)*floor(n/2),
 *               T(1) = 0.
 * Operação elementar contada: cada comparação de distância entre dois
 * elementos na etapa de combinação. A contagem independe dos valores.
 *
 * O custo da combinação domina o das chamadas recursivas, o que caracteriza
 * o Caso 3 do Teorema Mestre.
 */

#include <stdlib.h>
#include "comum.h"

static long long valor_absoluto(long long x) { return x < 0 ? -x : x; }

/* Devolve a menor distância entre dois elementos de v[ini..fim). */
long long par_mais_proximo(const long long *v, size_t ini, size_t fim) {
    size_t meio, i, j;
    long long melhor, esquerda, direita, distancia;
```

```
26
27      if (fim - ini < 2) return -1;        /* caso base: nenhum par existe */
28
29      meio = ini + (fim - ini) / 2;
30
31      esquerda = par_mais_proximo(v, ini, meio);
32      direita  = par_mais_proximo(v, meio, fim);
33
34      melhor = -1;
35      if (esquerda >= 0) melhor = esquerda;
36      if (direita >= 0 && (melhor < 0 || direita < melhor)) melhor = direita;
37
38      /* Combinação: todos os pares com um elemento em cada metade. */
39      for (i = ini; i < meio; i++)
40          for (j = meio; j < fim; j++) {
41              operacoes++;
42              distancia = valor_absoluto(v[i] - v[j]);
43              if (melhor < 0 || distancia < melhor) melhor = distancia;
44          }
45
46      return melhor;
47  }
```

src/main.c — execução dos experimentos

```
1   /* main.c  executa os experimentos dos três problemas e registra cada
2    * execução no mesmo arquivo CSV.
3    *
4    * Uso:  ./experimentos [arquivo.csv] [repeticoes]
5    *
6    * Cada problema é executado para uma sequência de tamanhos de entrada; para
7    * cada tamanho, o programa repete a execução e registra o tempo mediano,
8    * o número de operações elementares contadas e o valor previsto pela forma
9    * fechada demonstrada no relatório.
10   */
11
12  #include <stdio.h>
13  #include <stdlib.h>
14  #include <math.h>
15  #include "comum.h"
16
17  void karatsuba(const long long *p, const long long *q, size_t n, long long *res);
18  void multiplicar_direto(const long long *p, const long long *q, size_t n, long long *res);
19  void mergesort3(int *v, size_t n);
20  long long par_mais_proximo(const long long *v, size_t ini, size_t fim);
21
22  #define MAX_REP 64
23
24  /* Gerador pseudoaleatório próprio, para que as entradas sejam idênticas em
25     qualquer máquina ou compilador. */
26  static unsigned long long semente = 20261029ULL;
27  static unsigned long long aleatorio(void) {
28      semente = semente * 6364136223846793005ULL + 1442695040888963407ULL;
29      return semente >> 33;
30  }
31  static void reiniciar_semente(void) { semente = 20261029ULL; }
32
33  /* Previsões das formas fechadas demonstradas no relatório. */
34
35  /* P1: T(n) = (21*3^k - 24*2^k + 5)/2, com n = 2^k. */
36  static unsigned long long previsto_p1(size_t n) {
37      unsigned long long tres = 1, dois = 1;
38      while (dois < n) { tres *= 3; dois *= 2; }
39      return (21 * tres - 24 * dois + 5) / 2;
40  }
41
42  /* P1 direto: 2*n^2. */
43  static unsigned long long previsto_p1_direto(size_t n) {
44      return 2ULL * n * n;
45  }
46
```

```c
/* P2: T(n) = n * log_3(n), com n = 3^k. */
static unsigned long long previsto_p2(size_t n) {
    unsigned long long k = 0, p = 1;
    while (p < n) { p *= 3; k++; }
    return (unsigned long long)n * k;
}

/* P3: T(n) = n(n-1)/2. */
static unsigned long long previsto_p3(size_t n) {
    return (unsigned long long)n * (n - 1) / 2;
}

static void experimento_p1(const char *csv, int reps) {
    size_t tamanhos[] = { 2, 4, 8, 16, 32, 64, 128, 256, 512, 1024, 2048, 4096 };
    size_t qtd = sizeof(tamanhos) / sizeof(tamanhos[0]), t, i;
    int r;

    for (t = 0; t < qtd; t++) {
        size_t n = tamanhos[t];
        long long *p = malloc(n * sizeof(long long));
        long long *q = malloc(n * sizeof(long long));
        long long *res = malloc((2 * n) * sizeof(long long));
        double tempos[MAX_REP];
        unsigned long long ops = 0;

        reiniciar_semente();
        for (i = 0; i < n; i++) p[i] = (long long)(aleatorio() % 1000);
        for (i = 0; i < n; i++) q[i] = (long long)(aleatorio() % 1000);

        for (r = 0; r < reps; r++) {
            double t0 = agora_segundos();
            operacoes = 0;
            karatsuba(p, q, n, res);
            tempos[r] = agora_segundos() - t0;
            ops = operacoes;
        }
        registrar(csv, 1, "karatsuba", "T(n) = 3T(n/2) + 6n - 5", n, ops,
                  previsto_p1(n), mediana(tempos, reps), reps);

        if (n <= 2048) {
            for (r = 0; r < reps; r++) {
                double t0 = agora_segundos();
                operacoes = 0;
                multiplicar_direto(p, q, n, res);
                tempos[r] = agora_segundos() - t0;
                ops = operacoes;
            }
            registrar(csv, 1, "direto", "T(n) = 2n^2 (sem recursao)", n, ops,
                      previsto_p1_direto(n), mediana(tempos, reps), reps);
        }
        free(p); free(q); free(res);
    }
}

static void experimento_p2(const char *csv, int reps) {
    size_t tamanhos[] = { 3, 9, 27, 81, 243, 729, 2187, 6561, 19683, 59049,
                          177147, 531441 };
    size_t qtd = sizeof(tamanhos) / sizeof(tamanhos[0]), t, i;
    int r;

    for (t = 0; t < qtd; t++) {
        size_t n = tamanhos[t];
        int *original = malloc(n * sizeof(int));
        int *v = malloc(n * sizeof(int));
        double tempos[MAX_REP];
        unsigned long long ops = 0;

        reiniciar_semente();
        for (i = 0; i < n; i++) original[i] = (int)(aleatorio() % 1000000);

        for (r = 0; r < reps; r++) {
            double t0;
            for (i = 0; i < n; i++) v[i] = original[i];
```

```
            t0 = agora_segundos();
            operacoes = 0;
            mergesort3(v, n);
            tempos[r] = agora_segundos() - t0;
            ops = operacoes;
        }
        for (i = 1; i < n; i++)
            if (v[i - 1] > v[i]) {
                fprintf(stderr, "ERRO: vetor nao ordenado para n = %zu\n", n);
                exit(EXIT_FAILURE);
            }
        registrar(csv, 2, "mergesort3", "T(n) = 3T(n/3) + n", n, ops,
                  previsto_p2(n), mediana(tempos, reps), reps);
        free(original); free(v);
    }
}

static void experimento_p3(const char *csv, int reps) {
    size_t tamanhos[] = { 2, 4, 8, 16, 32, 64, 128, 256, 512, 1024, 2048,
                          4096, 8192 };
    size_t qtd = sizeof(tamanhos) / sizeof(tamanhos[0]), t, i;
    int r;

    for (t = 0; t < qtd; t++) {
        size_t n = tamanhos[t];
        long long *v = malloc(n * sizeof(long long));
        double tempos[MAX_REP];
        unsigned long long ops = 0;

        reiniciar_semente();
        for (i = 0; i < n; i++) v[i] = (long long)(aleatorio() % 100000000ULL);

        for (r = 0; r < reps; r++) {
            double t0 = agora_segundos();
            operacoes = 0;
            par_mais_proximo(v, 0, n);
            tempos[r] = agora_segundos() - t0;
            ops = operacoes;
        }
        registrar(csv, 3, "par_mais_proximo", "T(n) = 2T(n/2) + n^2/4", n, ops,
                  previsto_p3(n), mediana(tempos, reps), reps);
        free(v);
    }
}

int main(int argc, char *argv[]) {
    const char *csv = (argc >= 2) ? argv[1] : "resultados/resultados.csv";
    int reps = (argc >= 3) ? atoi(argv[2]) : 15;

    if (reps < 1 || reps > MAX_REP) reps = 15;

    printf("Executando os experimentos (%d repeticoes por ponto)...\n", reps);
    printf("  Problema 1: Karatsuba (Caso 1)\n");
    experimento_p1(csv, reps);
    printf("  Problema 2: MergeSort ternario (Caso 2)\n");
    experimento_p2(csv, reps);
    printf("  Problema 3: Par mais proximo (Caso 3)\n");
    experimento_p3(csv, reps);
    printf("Resultados gravados em %s\n", csv);
    return EXIT_SUCCESS;
}
```

B Arquivo de resultados

Todas as execuções foram registradas no arquivo `resultados/resultados.csv`, cujo conteúdo é reproduzido a seguir.

```
problema,algoritmo,recorrencia,n,operacoes,previsto,diferenca,tempo_mediano_s,repeticoes
1,karatsuba,"T(n) = 3T(n/2) + 6n - 5",2,10,10,0,0.000000098,15
1,direto,"T(n) = 2n^2 (sem recursao)",2,8,8,0,0.000000049,15
1,karatsuba,"T(n) = 3T(n/2) + 6n - 5",4,49,49,0,0.000000283,15
```

```
1,direto,"T(n) = 2n^2 (sem recursao)",4,32,32,0,0.000000073,15
1,karatsuba,"T(n) = 3T(n/2) + 6n - 5",8,190,190,0,0.000000905,15
1,direto,"T(n) = 2n^2 (sem recursao)",8,128,128,0,0.000000200,15
1,karatsuba,"T(n) = 3T(n/2) + 6n - 5",16,661,661,0,0.000002811,15
1,direto,"T(n) = 2n^2 (sem recursao)",16,512,512,0,0.000000708,15
1,karatsuba,"T(n) = 3T(n/2) + 6n - 5",32,2170,2170,0,0.000008764,15
1,direto,"T(n) = 2n^2 (sem recursao)",32,2048,2048,0,0.000002765,15
1,karatsuba,"T(n) = 3T(n/2) + 6n - 5",64,6889,6889,0,0.000026815,15
1,direto,"T(n) = 2n^2 (sem recursao)",64,8192,8192,0,0.000010866,15
1,karatsuba,"T(n) = 3T(n/2) + 6n - 5",128,21430,21430,0,0.000082209,15
1,direto,"T(n) = 2n^2 (sem recursao)",128,32768,32768,0,0.000042074,15
1,karatsuba,"T(n) = 3T(n/2) + 6n - 5",256,65821,65821,0,0.000239998,15
1,direto,"T(n) = 2n^2 (sem recursao)",256,131072,131072,0,0.000166073,15
1,karatsuba,"T(n) = 3T(n/2) + 6n - 5",512,200530,200530,0,0.000770285,15
1,direto,"T(n) = 2n^2 (sem recursao)",512,524288,524288,0,0.000704104,15
1,karatsuba,"T(n) = 3T(n/2) + 6n - 5",1024,607729,607729,0,0.002247680,15
1,direto,"T(n) = 2n^2 (sem recursao)",1024,2097152,2097152,0,0.002655534,15
1,karatsuba,"T(n) = 3T(n/2) + 6n - 5",2048,1835470,1835470,0,0.008165742,15
1,direto,"T(n) = 2n^2 (sem recursao)",2048,8388608,8388608,0,0.010439143,15
1,karatsuba,"T(n) = 3T(n/2) + 6n - 5",4096,5530981,5530981,0,0.020226787,15
2,mergesort3,"T(n) = 3T(n/3) + n",3,3,3,0,0.000000053,15
2,mergesort3,"T(n) = 3T(n/3) + n",9,18,18,0,0.000000115,15
2,mergesort3,"T(n) = 3T(n/3) + n",27,81,81,0,0.000000372,15
2,mergesort3,"T(n) = 3T(n/3) + n",81,324,324,0,0.000001336,15
2,mergesort3,"T(n) = 3T(n/3) + n",243,1215,1215,0,0.000004473,15
2,mergesort3,"T(n) = 3T(n/3) + n",729,4374,4374,0,0.000015307,15
2,mergesort3,"T(n) = 3T(n/3) + n",2187,15309,15309,0,0.000097881,15
2,mergesort3,"T(n) = 3T(n/3) + n",6561,52488,52488,0,0.000410988,15
2,mergesort3,"T(n) = 3T(n/3) + n",19683,177147,177147,0,0.001432884,15
2,mergesort3,"T(n) = 3T(n/3) + n",59049,590490,590490,0,0.004885286,15
2,mergesort3,"T(n) = 3T(n/3) + n",177147,1948617,1948617,0,0.016352706,15
2,mergesort3,"T(n) = 3T(n/3) + n",531441,6377292,6377292,0,0.049505573,15
3,par_mais_proximo,"T(n) = 2T(n/2) + n^2/4",2,1,1,0,0.000000036,15
3,par_mais_proximo,"T(n) = 2T(n/2) + n^2/4",4,6,6,0,0.000000045,15
3,par_mais_proximo,"T(n) = 2T(n/2) + n^2/4",8,28,28,0,0.000000077,15
3,par_mais_proximo,"T(n) = 2T(n/2) + n^2/4",16,120,120,0,0.000000183,15
3,par_mais_proximo,"T(n) = 2T(n/2) + n^2/4",32,496,496,0,0.000000558,15
3,par_mais_proximo,"T(n) = 2T(n/2) + n^2/4",64,2016,2016,0,0.000001990,15
3,par_mais_proximo,"T(n) = 2T(n/2) + n^2/4",128,8128,8128,0,0.000007618,15
3,par_mais_proximo,"T(n) = 2T(n/2) + n^2/4",256,32640,32640,0,0.000029646,15
3,par_mais_proximo,"T(n) = 2T(n/2) + n^2/4",512,130816,130816,0,0.000119979,15
3,par_mais_proximo,"T(n) = 2T(n/2) + n^2/4",1024,523776,523776,0,0.000481571,15
3,par_mais_proximo,"T(n) = 2T(n/2) + n^2/4",2048,2096128,2096128,0,0.001911846,15
3,par_mais_proximo,"T(n) = 2T(n/2) + n^2/4",4096,8386560,8386560,0,0.008183062,15
3,par_mais_proximo,"T(n) = 2T(n/2) + n^2/4",8192,33550336,33550336,0,0.032548368,15
```